# OPERATING SYSTEMS – ASSIGNMENT 1

**Name:** S. Sharvesh **Register No:** 192525458

# TASK 6 – Evaluate the Cost of System Calls (4 Marks)

```
Evaluate why each system call costs more than an ordinary function call; how buffer size changes read() and write() calls; h
Complete the Code: Complete TODO 1 in the supplied copy_file program using read() and write(). Test buffer sizes 1, 512 and
Alternative Concept and Complexity Analysis: Evaluate mmap instead of buffered read()/write(), including system calls, copy
```

Evidence Expected: Measured system-call counts and times for different buffer sizes.

## ANSWER

A normal function call usually changes control flow within the same process and privilege level. A system call crosses the user/kernel boundary, saves/restores processor state, validates arguments and permissions, may acquire kernel locks, may block, and may trigger filesystem, device or scheduler work. Therefore, system calls have a larger constant cost than ordinary function calls. The cost is not a fixed number: cache state, filesystem cache hits, device latency, contention and kernel version affect it.

For n bytes and buffer size b, the direct-read call count in this program is ceil(n/b) + 1 because it performs one final read that returns zero at end-of-file. Thus the call count is O(n/b), while data-copy time is O(n). Increasing b reduces system-call overhead but uses more user-space buffer memory. A practical backup buffer should be benchmarked; 64 KiB to 1 MiB is a reasonable range to test, but the fastest size depends on storage, cache and workload.

```
6.1 Complete Code – TODO 1 (copy_file using read() and write())
```

#define _POSIX_C_SOURCE 200809L

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <errno.h>
#include <stdlib.h>
```

long copy_file(const char *src, const char *dst, size_t bufsize)

```
{
```

char buf[8192];

if (bufsize == 0 || bufsize > sizeof(buf)) {

errno = EINVAL;

```
return -1;
}
```

int in = open(src, O_RDONLY);

```
if (in < 0) return -1;
```

int out = open(dst, O_WRONLY | O_CREAT | O_TRUNC, 0644);

if (out < 0) {

close(in);

```
return -1;
}
```

long calls = 0;

ssize_t n;

for (;;) {

```
do {
  n = read(in, buf, bufsize);
calls++;
  } while (n < 0 && errno == EINTR);
if (n == 0) break;
if (n < 0) {
calls = -1;
break;
  }
ssize_t written = 0;
while (written < n) {
  ssize_t w = write(out, buf + written, (size_t)(n - written));
if (w < 0 && errno == EINTR) continue;
if (w <= 0) {
calls = -1;
break;
  }
written += w;
  }
if (calls < 0) break;
  }
int saved_errno = errno;
if (close(in) < 0 && calls >= 0) calls = -1;
if (close(out) < 0 && calls >= 0) calls = -1;
if (calls < 0) errno = saved_errno;
  return calls;
  }
  int main(void)
  {
FILE *f = fopen("records.dat", "w");
  if (!f) { perror("fopen"); return 1; }
for (int i = 0; i < 10000; i++) fputc('A' + i % 26, f);
  if (fclose(f) != 0) { perror("fclose"); return 1; }
size_t sizes[] = {1, 512, 4096};
for (int i = 0; i < 3; i++) {
long result = copy_file("records.dat", "copy.dat", sizes[i]);
if (result < 0) perror("copy_file");
  else printf("buffer %4zu bytes: %5ld read() calls\n",
sizes[i], result);
  }
unlink("records.dat");
```

unlink("copy.dat");

```
return 0;
}
```

6.2 Compile and execute

gcc -std=c11 -Wall -Wextra -O2 copy_file.c -o copy_file

./copy_file

strace -c ./copy_file

strace -o copy_trace.txt -e trace=read,write,openat,close ./copy_file

Expected program output (as specified in the assignment):

```
buffer    1 bytes: 10001 read() calls
buffer  512 bytes:    21 read() calls
buffer 4096 bytes:     4 read() calls
strace -c counts all read calls made by the process and its runtime, not necessarily only the calls counted inside copy_fil
```

6.3 Alternative: memory-mapped files (mmap)

With mmap, the process maps file pages into its virtual address space and accesses them as memory. It may need only a small number of setup/teardown system calls (open, fstat, mmap, munmap, close), rather than about n/b read calls and n/b write calls. The kernel loads pages on demand through page faults. mmap can avoid an explicit user-buffer copy for reads, but dirty mapped pages still need write-back and page faults have costs. For very large files, mapping the entire file consumes a large virtual address range and page tables; mapping windows limits address-space and page-table pressure. Both methods use O(n) data space in the file/page cache in the broad system sense, while a buffered copy uses O(b) application buffer space. mmap is not automatically faster for sequential backup; benchmark it on the target storage.

## QUESTION FROM THE PROVIDED ASSIGNMENT

Buffer size | Data read calls for 10,000 bytes | EOF read | Total expected read() calls

1 byte | 10,000 | 1 | 10,001

512 bytes | 20 | 1 | 21

4096 bytes | 3 | 1 | 4